Practice Roadmap: GREEDY

Generated: 2026-08-19 23:57:37

Rating Range: 300 to 800

Curated Problems

• 1. [800] 2182A - New Year String (URL: https://codeforces.com/problemset/problem/2182/A)

• 2. [800] 2140A - Shift Sort (URL: https://codeforces.com/problemset/problem/2140/A)

• 3. [800] 2179B - Blackslex and Showering (URL: https://codeforces.com/problemset/problem/2179/B)

• 4. [800] 1517A - Sum of 2050 (URL: https://codeforces.com/problemset/problem/1517/A)

Roadmap & Strategy

Here is a structured practice roadmap for the **Greedy** topic using your specified 800-rated problems.

---

1. Core Concepts to Learn

Before diving into the problems, master these fundamental greedy concepts at the 800-rating level:

• 1. **The Greedy Choice Property**: Making the best *local* decision at each step to reach a *global* optimal solution without needing to backtrack.

• 2. **Structural Reduction / Base-K Logic**: Reducing a problem by repeatedly subtracting or taking the largest possible power/multiple available (similar to the standard coin-change problem when denominations are clean).

• 3. **Sorting as Preprocessing**: Often, sorting elements numerically or chronologically makes the optimal greedy decision obvious.

• 4. **Interval / Gap Analysis**: Finding maximal continuous spaces (e.g., time intervals between events) by evaluating adjacent elements.

• 5. **Constructive Invariants**: Building a sequence step-by-step while maintaining specific conditions (e.g., avoiding forbidden substrings or minimizing shifts).

---

2. Recommended Order to Solve & Rationale

Step 1: 1517A - Sum of 2050

• **Tags**: `greedy`, `math`

• **Why start here?**

• * This problem introduces the classic **"largest choice first"** greedy pattern in its purest math form.

• * You need to represent N as a sum of numbers of the form 2050×10^k.

• * **Key Insight**: If N is not divisible by 2050, answer is `-1`. Otherwise, the minimum number of terms required equals the **sum of digits** of $N / 2050$.

Step 2: 2179B - Blackslex and Showering

• **Tags**: `dp`, `greedy`, `implementation`

• **Why second?**

• * Transitions from pure math to **array/interval checking**.

• * You are checking whether a contiguous free time slot of length S exists within a set of interval constraints (or maximizing available continuous space).

• * **Key Insight**: Check the gap before the first event, the gap after the last event, and all gaps between consecutive events. The optimal choice is simply the maximum gap found.

Step 3: 2140A - Shift Sort

• **Tags**: `greedy`, `sortings`

• **Why third?**

• * Teaches you how **sorting directly guides greedy operations** on arrays.

• * Typically involves counting how many elements are already in their correct sorted position versus those that need to be moved/shifted.

• * **Key Insight**: Find what remains invariant or count elements that *do not* need to move to minimize the required shift operations.

Step 4: 2182A - New Year String

• **Tags**: `constructive algorithms`, `greedy`, `implementation`, `strings`

• **Why last?**

• * Combines greedy decision-making with **string manipulation and constructive logic**.

• * Requires modifying or building a string character-by-character to satisfy a condition with minimum operations.

• * **Key Insight**: Process the string sequentially and make local replacements immediately whenever a target pattern or conflict is detected.

---

3. Common Patterns

• **Largest Element First (Coin Change Pattern)**:

• * When multiples/powers are well-behaved (like powers of 10), greedily picking the largest possible value never prevents you from reaching the optimal total (e.g., *1517A*).

• **Adjacent Pair / Interval Checking**:

• * Instead of evaluating all pairs ($O(N^2)$), sort or linearly scan adjacent elements (A_i, A_{i+1}) to compute local gaps or transitions (e.g., *2179B*).

• **Matching against Sorted Target**:

• * Compare the current sequence against its sorted version to greedily count mismatched blocks or misaligned elements (e.g., *2140A*).

---

4. Common Mistakes

• 1. **Integer Overflow**:

• * *1517A* uses inputs up to 10^{18}. Using a standard 32-bit `int` will cause overflow. Always use `long long` in C++ / standard integers in Python.

• 2. **Applying Greedy Without Proving Feasibility**:

• * Assuming a greedy choice works without checking edge cases (e.g., forgetting to output `-1` when N cannot be divided by 2050).

• 3. **Off-by-One Errors in Boundaries**:

• * In gap/interval problems (*2179B*), beginners often forget the boundary gaps: the space before the very first event (0 to L_1) and the space after the very last event (R_n to total time M).

• 4. **Overcomplicating with Dynamic Programming**:

• * Although *2179B* has a `dp` tag, 800-rated problems rarely require actual DP tables. Avoid overthinking—look for simple linear $O(N)$ or $O(N \log N)$ greedy scanning first.